

★ WEEK 14 · PART 3 of 3

Hamiltonian Monte Carlo and NUTS

Roll a Ball, Don't Wander

The physical picture · leapfrog · the U-turn · the modern default

A random walk explores blindly. HMC treats the parameter as a ball rolling on a landscape shaped by the posterior — one kick carries it far in a smart direction. NUTS decides how far to roll, automatically.

3

PART 3 · HMC & NUTS

The Sampler That Rolls

Why random walks are slow, the physics of Hamiltonian Monte Carlo, leapfrog integration, the U-turn problem, and how NUTS turns HMC into the default engine behind modern tools.

We Can Sample — But Slowly

Parts 1-2 built the chain; can we do better?

1

Metropolis-Hastings works anywhere

Propose a nearby move, accept by a ratio. Simple and general — but it wanders like a blindfolded walker.

2

Random walks are slow in high dimensions

With many parameters, tiny aimless steps take forever to explore the posterior. Draws stay highly correlated.

3

HMC uses physics to move smartly

Treat the parameter as a rolling ball guided by the posterior's shape. One move travels far — and NUTS tunes how far.

Part 3 in one line: HMC rolls a ball across the posterior; NUTS decides when to stop rolling.

The Trouble With Random Walks

Aimless steps waste effort

A random-walk sampler proposes blindly — a small step in a random direction, with no sense of where the posterior is. In high dimensions almost every blind direction is wrong, so it inches along and its draws stay heavily correlated.

Why it struggles

- Each proposal ignores the shape of the target.
- Big steps get rejected; small steps barely move.
- In many dimensions, random directions rarely help.

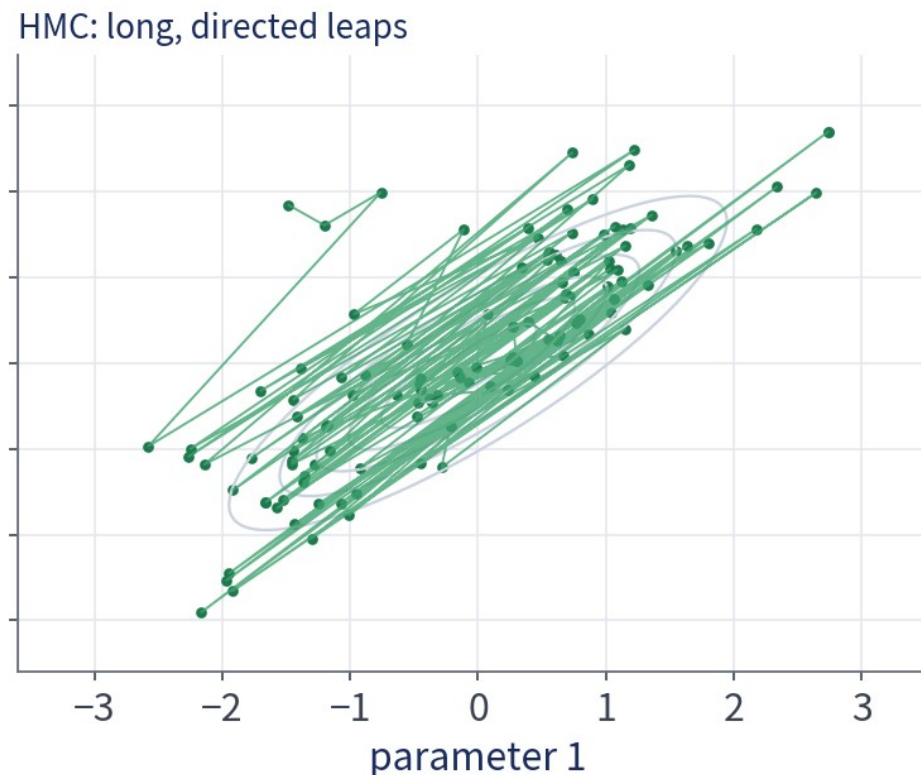
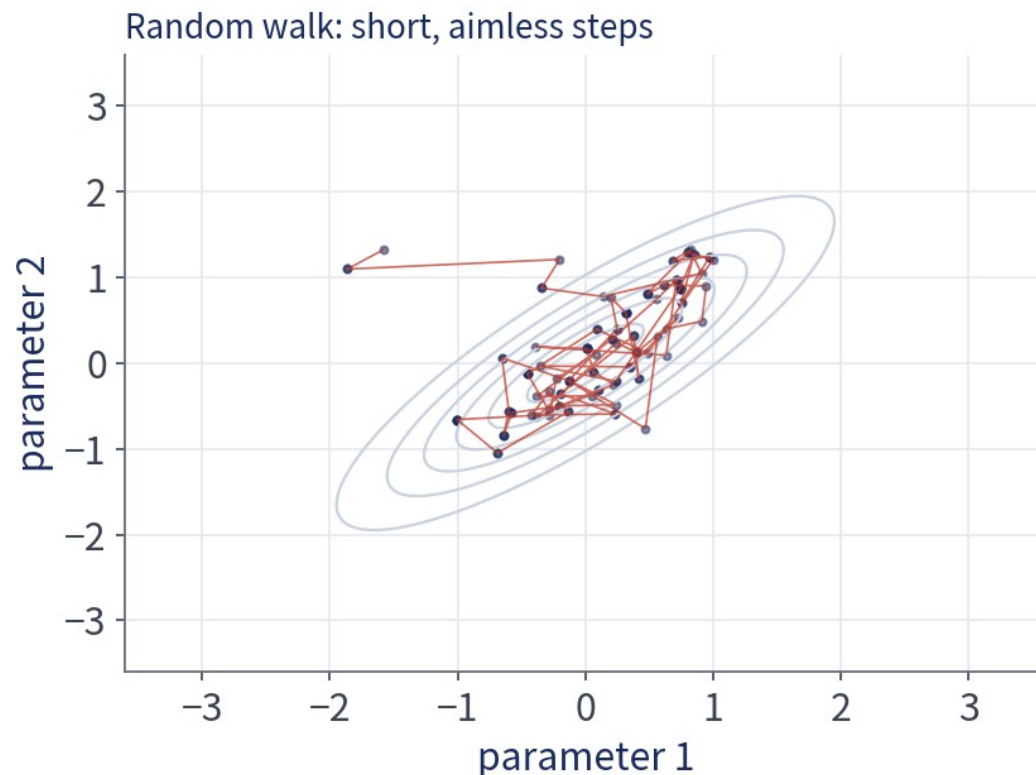
The cost

- The chain explores painfully slowly.
- Consecutive draws are highly correlated.
- You need enormous chains for decent precision.

Random Walk vs HMC

The same number of steps on the same target. The random walk (left) shuffles around in short, aimless hops; HMC (right) makes long, directed leaps that cover the whole distribution quickly.

Same number of steps: HMC covers the distribution far faster



Roll a Ball, Don't Wander

Borrow the physics of motion

HMC's insight is physical. Picture the posterior flipped into a valley — high posterior is low ground. Place a ball, give it a random kick, and let it roll. Gravity pulls it toward the low ground, so it naturally visits high-posterior regions.

The mental model

- The parameter is a ball on a landscape.
- The landscape is the posterior, turned upside down.
- A random kick sets it rolling; you follow the path.

Why it is smart

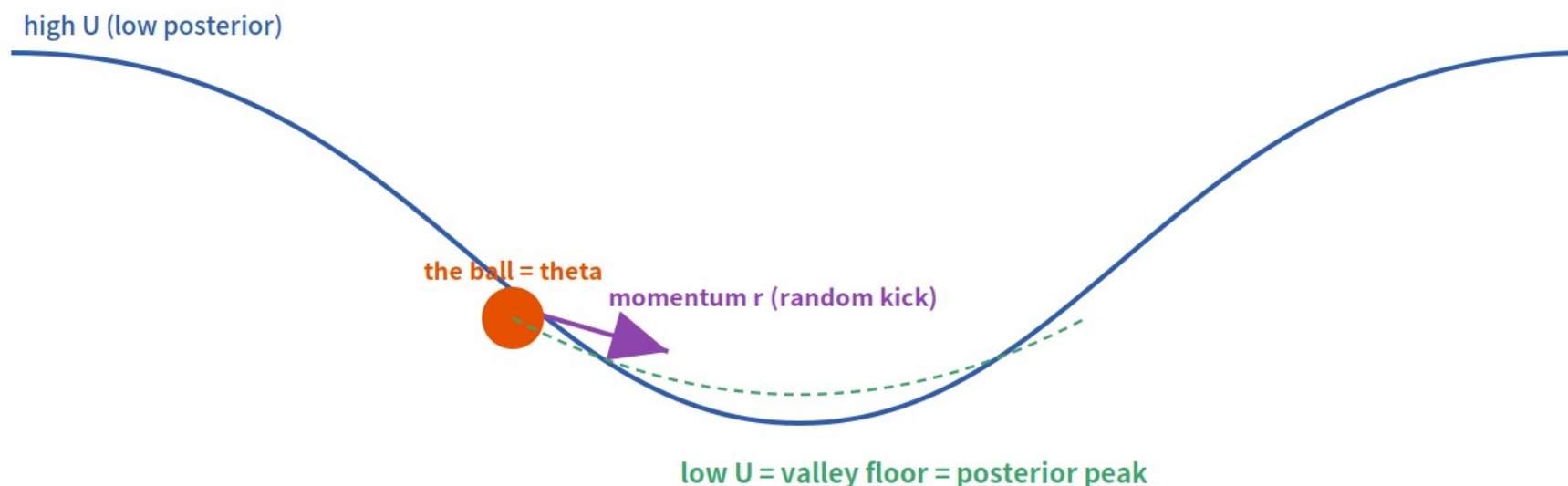
- The ball uses the slope to move in useful directions.
- One roll carries it far across the distribution.
- No blind guessing — the geometry guides it.

The Physical Picture

Flip the posterior into a valley: its peak becomes the valley floor. The ball is the parameter, the arrow is a random momentum, and the slope of the landscape pulls the ball toward high-posterior ground.

The Physical Picture

Turn the posterior upside-down into a valley, then roll a ball across it.



The ball is pulled downhill by the SLOPE of the landscape (the gradient) - so it naturally heads toward high-posterior regions.

Potential Energy = Negative Log Posterior

The shape the ball rolls on

$$\underbrace{U(\theta)}_{\text{potential energy}} = -\log p(\theta \mid \text{data}) \quad (\text{the landscape the ball rolls on})$$

What it means

- Take the log of the posterior and flip its sign.
- High posterior becomes low 'ground' — a valley.
- This surface is the potential energy.

The intuition

- The ball is pulled toward low potential.
- Low potential means high posterior.
- So physics steers it exactly where we want draws.

Adding Momentum

A random push to get things moving

The momentum variable

- We invent an extra variable: the ball's momentum.
- At each step we draw it randomly from a Gaussian.
- It is the kick that starts the ball rolling.

Kinetic energy

- Motion carries kinetic energy — one half the momentum squared.
- It trades off against the potential as the ball moves.
- Uphill, the ball slows; downhill, it speeds up.

The Hamiltonian

Potential plus kinetic, and it barely changes

$$\underbrace{H(\theta, r)}_{\text{total energy}} = \underbrace{U(\theta)}_{\text{potential}} + \underbrace{\frac{1}{2} r^\top r}_{\text{kinetic}} \text{ stays (nearly) constant as the ball rolls}$$

What it is

- The total energy: potential plus kinetic.
- As the ball rolls, energy shifts between the two.
- But the TOTAL stays (almost) constant.

Why that matters

- Constant energy means the proposal is on the right level set.
- So the far-away endpoint is a valid posterior draw.
- That is what makes long moves get accepted.

Why Energy Conservation Helps

Long moves that still get accepted

The random-walk dilemma

- Big proposals move far but are usually rejected.
- Small proposals are accepted but barely move.
- You seem stuck choosing between the two.

How HMC escapes it

- The ball travels far by following the landscape.
- Conserved energy keeps the endpoint just as probable.
- So you get BOTH big moves AND high acceptance.

Following the Gradient

The ball needs to feel the slope

What the ball uses

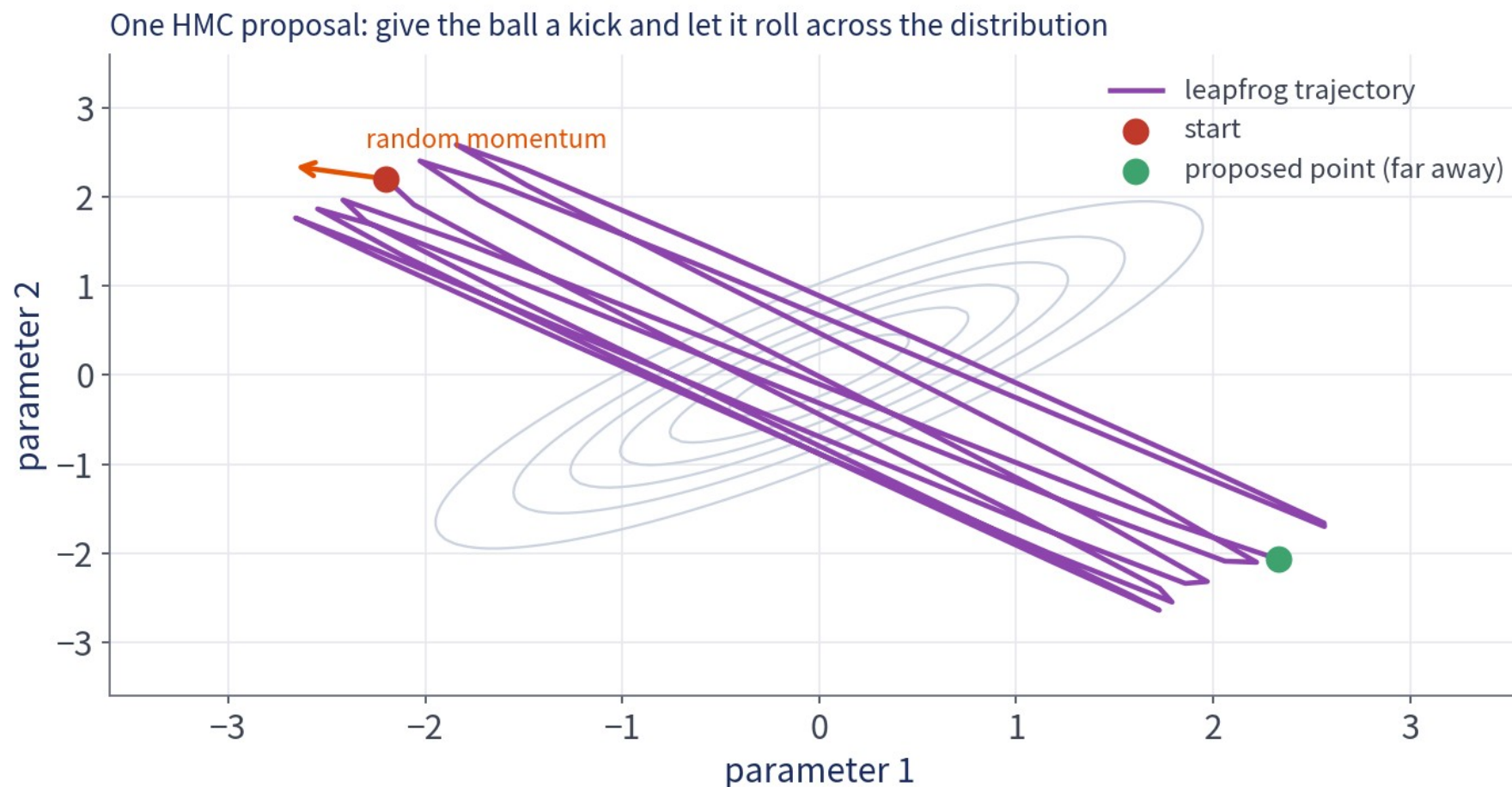
- The slope of the landscape — the gradient.
- It points in the direction the ball should roll.
- This is what makes the moves purposeful, not blind.

The catch

- You must be able to compute that gradient.
- So the model has to be differentiable.
- Modern tools get the gradient automatically.

One HMC Proposal Travels Far

Give the ball a random momentum (orange) and let it roll along the landscape. A single trajectory sweeps clear across the distribution, ending at a distant point that is still a valid posterior draw.



Leapfrog Integration

How the computer rolls the ball

A computer can't roll the ball continuously — it takes small steps, just like discretizing an SDE. The leapfrog method does this in a special order that keeps energy almost perfectly conserved, which is exactly what HMC needs.

The idea

- Advance momentum and position in tiny steps.
- Interleave them: half a momentum step, a full position step, half again.
- Repeat for a chosen number of steps.

Why this order

- It is reversible and volume-preserving.
- So it tracks the true motion with tiny energy drift.
- That near-constant energy keeps acceptance high.

The Leapfrog Step

Half-kick, drift, half-kick

$$\text{leapfrog: } r \leftarrow r - \frac{\varepsilon}{2} \nabla U(\theta); \quad \theta \leftarrow \theta + \varepsilon r; \quad r \leftarrow r - \frac{\varepsilon}{2} \nabla U(\theta)$$

Reading it

- First a half step of momentum, using the slope.
- Then a full step of position, carried by momentum.
- Then another half step of momentum.

The two dials

- The step size sets how fine each move is.
- The number of steps sets how far the ball rolls.
- Tuning them by hand is the awkward part — NUTS fixes it.

The Accept Step

A small correction for a small error

Why we still accept-reject

- Leapfrog is excellent but not perfect.
- The energy drifts by a tiny amount.
- A Metropolis step corrects for that drift.

What it guarantees

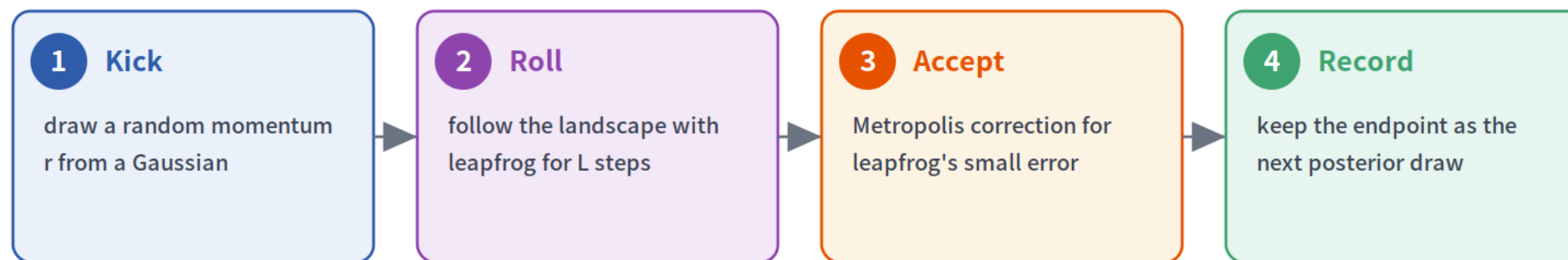
- The chain still targets the exact posterior.
- Because energy barely changes, acceptance is high.
- Rejections are rare, unlike in a random walk.

One HMC Step, End to End

Kick the ball with a random momentum, roll it with leapfrog for several steps, apply a Metropolis correction, and record the endpoint. Because energy is conserved, that distant point is almost always accepted

One HMC Step

Kick the ball, let it roll, then accept - and you have travelled far in one move.



Why this beats a random walk

Because the ball follows the landscape, one roll carries it a long way to a NEW high-posterior region - and because energy is nearly conserved, that far-away proposal is almost always accepted.

Long, purposeful moves with high acceptance - the opposite of aimless wandering.

How Long Should the Ball Roll?

The one dial HMC can't set itself

HMC has a weakness: you must choose how far to roll. Too short, and it behaves like a random walk again. Too long, and the ball loops around and comes back where it started — wasted effort. Getting this right by hand is fiddly.

Too short a roll

- The ball barely moves from where it started.
- You lose the whole advantage of HMC.
- It degenerates into a slow random walk.

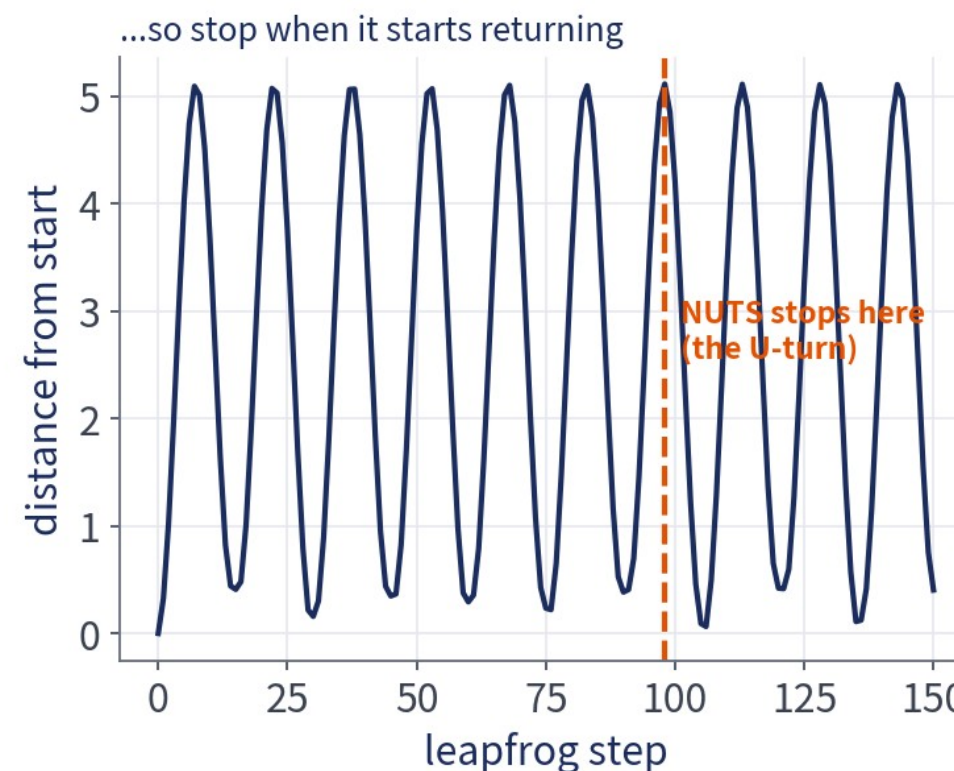
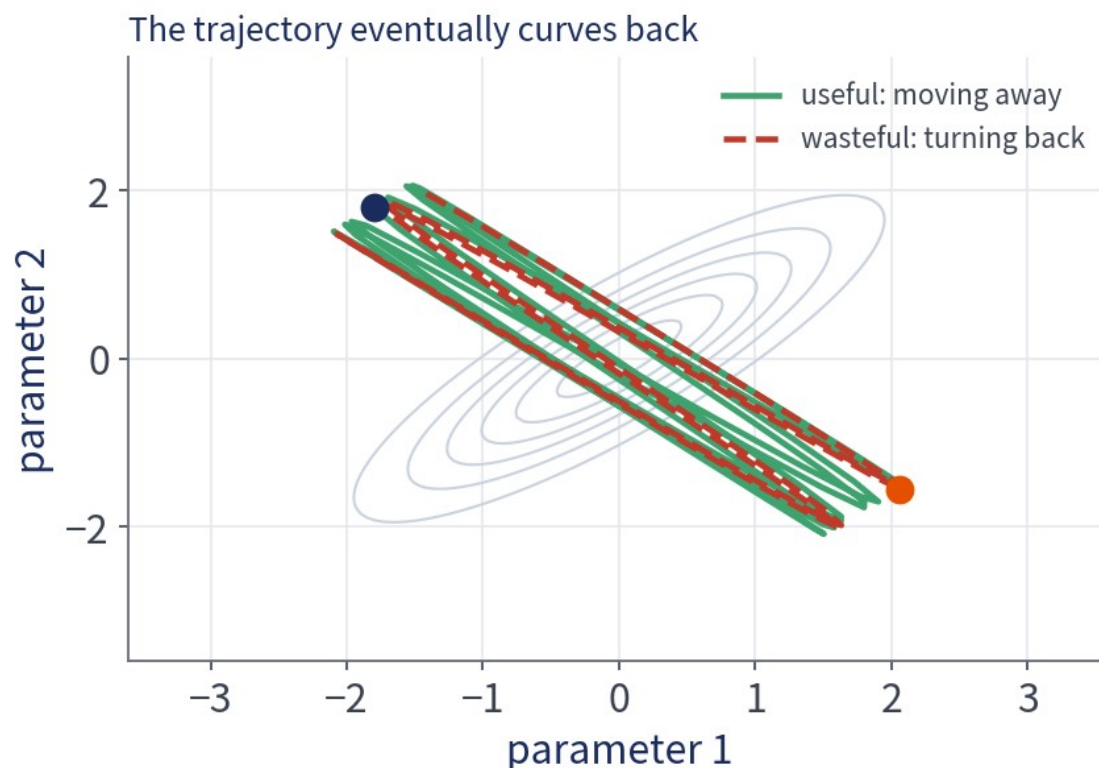
Too long a roll

- The ball curves around and heads back.
- The extra steps are wasted computation.
- It can even return near its starting point.

The U-Turn Problem

As the ball rolls, its distance from the start grows, then peaks, then shrinks as the trajectory curves back. That peak — the moment it begins to return — is the natural place to stop.

How far to roll? NUTS stops automatically at the U-turn



NUTS: Stop at the U-Turn

The No-U-Turn Sampler tunes the length for you

The No-U-Turn Sampler, NUTS, removes the guesswork. It keeps rolling the ball and simply stops the moment the trajectory starts to turn back on itself. No trajectory length to set — it finds the right one automatically, every step.

What NUTS does

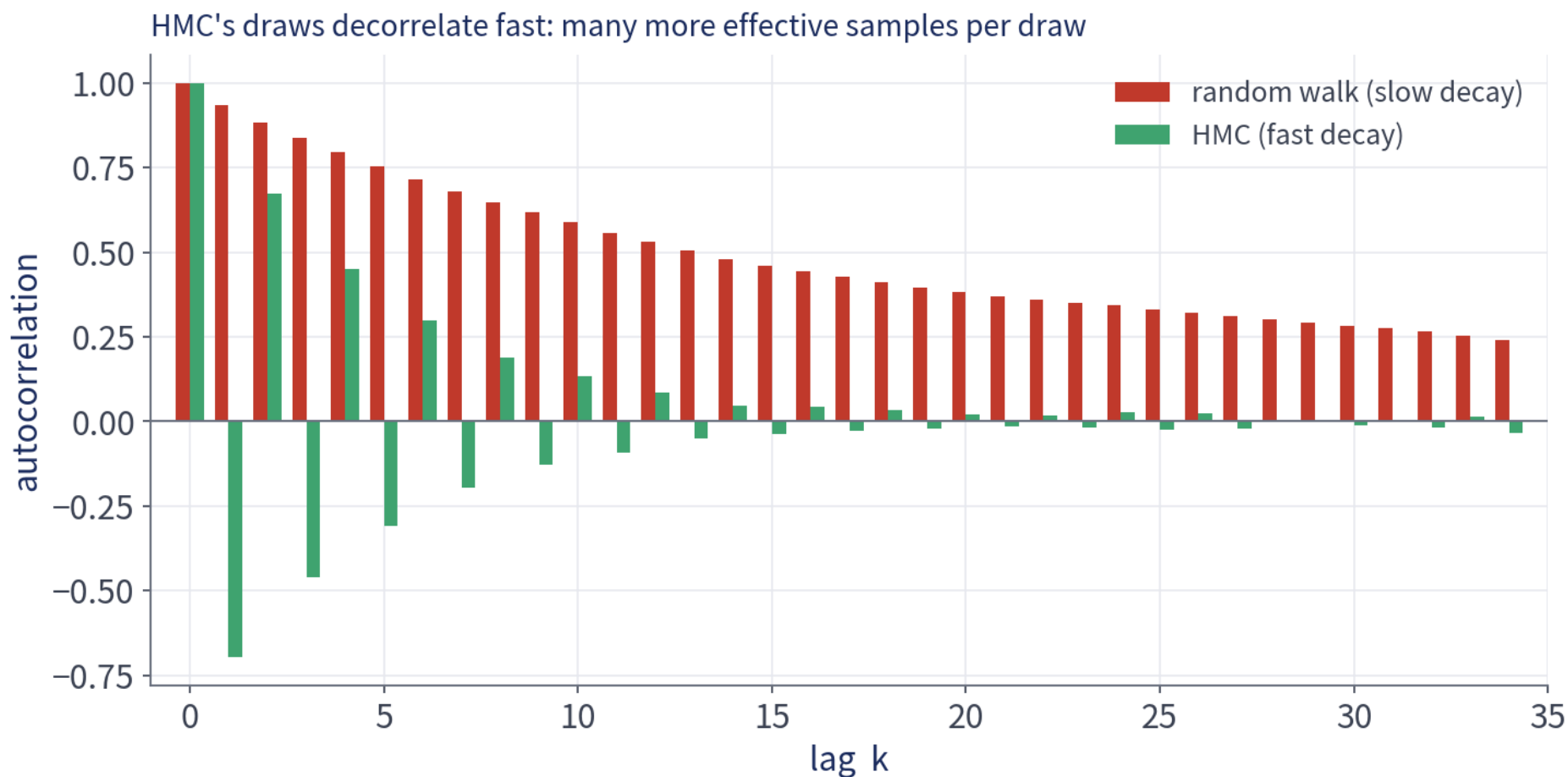
- Extend the trajectory until it starts to U-turn.
- Then stop — no manual length needed.
- It adapts the length at every iteration.

Why it is the default

- It combines HMC's speed with zero length-tuning.
- It just works across a huge range of models.
- It is the engine inside modern Bayesian tools.

Why It Wins: Fast Decorrelation

The random walk's draws stay correlated for many lags; HMC's decorrelate almost immediately. Fewer correlated draws means far more effective samples for the same effort — the practical payoff of rolling smartly.



This Is What PyMC and Stan Use

You get HMC without coding it

What you write

- Just the model: priors and a likelihood.
- The tool computes gradients for you.
- It runs NUTS and tunes the step and length.

What you get

- Efficient, low-correlation posterior draws.
- Far higher effective sample size per second.
- State-of-the-art sampling with almost no effort.

Roll the Ball Yourself

Direct Claude Code to build a tiny HMC

GOAL Code a minimal HMC and see it beat a random walk on the same target.

[INPUT] Prompt · Claude Code

Write and run `hmc_demo.py` that:

1. defines a 2D Gaussian target and its potential $U = -\log \text{posterior}$, plus its gradient;
2. implements leapfrog and a Hamiltonian Monte Carlo step (kick, roll L steps, accept);
3. also runs a random-walk Metropolis sampler on the same target;
4. compares them: overlay the sample paths and plot autocorrelation for each.

Show HMC decorrelates far faster and report each sampler's effective sample size.

How to work with it

- Potential $U = \text{minus log posterior}$.
- Leapfrog: half-kick, drift, half-kick.
- One roll should travel far.
- HMC's ESS should dwarf the random walk's.

Deliverable: `hmc_demo.py` + overlaid sample paths and autocorrelation for HMC vs random walk.

Three Questions Before You Finish

If you can answer these, Part 3 has landed

Q1 In HMC, what plays the role of the landscape the ball rolls on?

Negative log posterior

Q2 Why do HMC's long proposals still get accepted?

Energy is nearly conserved

Q3 What does NUTS decide automatically?

How far to roll (trajectory length)

All three come from the potential landscape, energy conservation, and the U-turn rule.

Three Misconceptions to Avoid

Where HMC intuition slips

Common myths

- "HMC is a totally different idea from Metropolis."
- "HMC works on any model, no matter what."
- "A longer trajectory is always better."

The reality

- It still ends with a Metropolis accept step.
- It needs a differentiable model to get the gradient.
- Too long U-turns and wastes effort — that's why NUTS exists.

Week 14 in Three Ideas

Pulling the whole week together

1

Sample, don't integrate

Parts 1-2: MCMC sidesteps the intractable normalizer by drawing samples.

Metropolis-Hastings proposes and accepts by a ratio; the histogram of draws IS the posterior.

2

HMC rolls instead of wandering

Part 3: treat the parameter as a ball on the negative-log-posterior landscape. A random kick plus a leapfrog roll carries it far in a useful direction, and conserved energy keeps acceptance high.

3

NUTS makes it automatic

The one dial HMC can't set — how far to roll — is handled by NUTS, which stops at the U-turn. This is why PyMC and Stan sample so efficiently with almost no tuning.

Next — Week 15: Bayesian state-space models and HMM, where sampling meets hidden states.